

Ingeniería de software

Primer parcial · Guía integrada con atlas UML y anotaciones de examen

Recorrido central: comprender el problema, modelar conceptos y comportamientos, asignar responsabilidades y construir una solución con alta cohesión, bajo acoplamiento y trazabilidad.

Ruta de lectura	Páginas
Fundamentos, procesos y análisis orientado a objetos	1-3
Requisitos, casos de uso y comportamiento UML	4-6
Relaciones UML, multiplicidades y SOLID	7-9
Catálogo y diseños UML de patrones	10-15
Arquitectura y caso Wonderland Tech	16-18
Glosario, autoevaluación y repaso	19-21

Naturaleza del software

El software **no se desgasta físicamente**. Los defectos y las modificaciones pueden deteriorar su funcionamiento o estructura. En la curva idealizada de fallas, la tasa inicial disminuye al corregir errores; en la curva real aparecen nuevos picos tras cambios con efectos colaterales.

Campo de aplicación	Finalidad
Sistemas	Dar soporte: sistemas operativos, compiladores y controladores.
Tiempo real	Responder a eventos dentro de plazos definidos.
Gestión	Procesar operaciones y datos del negocio.
Científico e ingeniería	Cálculo numérico, simulación y análisis.
Embebido	Control integrado en dispositivos y productos.
Computadoras personales	Ofimática, edición y herramientas locales.
Web	Aplicaciones apoyadas en navegadores y servidores.
Inteligencia artificial	Aprendizaje, inferencia y reconocimiento de patrones.

CONCEPTOS INDISPENSABLES

La corrección de un sistema de tiempo real depende del resultado **y del plazo**. Mantenimiento implica cambio; el cambio exige validar que el comportamiento siga siendo correcto.

Procesos y calidad

Pressman · Modelos de desarrollo · CMMI · Cleanroom

Proceso: estructura de actividades, acciones y tareas para construir y mantener software. Un patrón de proceso describe un problema recurrente de ejecución o gestión, su contexto y una solución probada.

Pressman: Comunicación → Planeación → Modelado → Construcción → Despliegue. Comprender requisitos; estimar recursos y riesgos; analizar y diseñar; programar y probar; entregar y recibir evaluación. El flujo puede ser lineal, iterativo o concurrente.

Modelo	Idea que lo distingue
Cascada	Etapas predominantemente secuenciales; cambios tardíos costosos. Análisis → Diseño → Desarrollo → Pruebas → Implementación → Mantenimiento.
DRA / RAD	Desarrollo rápido apoyado en componentes, herramientas y plazos breves.
Incremental	Entregas operativas sucesivas que agregan funcionalidad.
Prototipos	Representaciones tempranas para aclarar requisitos y expectativas.
Espiral	Ciclos guiados por evaluación explícita de riesgos.
Concurrente	Actividades simultáneas que se encuentran en estados diferentes.
Componentes	Integración de piezas reutilizables; exige evaluar compatibilidad.
Métodos formales	Especificación y verificación matemática de propiedades.
Orientado a aspectos	Separación de preocupaciones transversales, como auditoría.
Proceso Unificado	Iterativo, dirigido por casos de uso y centrado en arquitectura. RUP es una realización de UP.

Iterativo significa refinar en ciclos; **incremental** significa agregar capacidad. Un proceso puede combinar ambos. **Definición = qué; desarrollo = cómo; mantenimiento = cambio.**

Cuatro capas de la ingeniería (Pressman): compromiso con la calidad → proceso → métodos → herramientas (CASE, IDE y automatización). No son capas de una aplicación.

CMMI (SEI): Inicial → Administrado → Definido → Cuantitativamente administrado → Optimizado. Evalúa madurez de procesos; no garantiza ausencia de errores en el producto.

Cleanroom: caja negra = estímulos y respuestas; caja de estado = datos y transiciones internas; caja transparente = algoritmos que implementan esas transiciones.

CONCEPTOS INDISPENSABLES

Cascada organiza etapas; incremental organiza entregas; espiral prioriza riesgos. Calidad, proceso, métodos y herramientas son niveles complementarios.

Análisis y diseño OO

Del enunciado a los objetos de software

El **batche semántico** es la distancia entre los conceptos del problema y su representación en el software. ADOO busca reducirla mediante continuidad: conceptos del negocio → modelo de dominio → objetos de software.

Aspecto	Análisis OO	Diseño OO
Pregunta	¿Qué se necesita?	¿Cómo se resolverá?
Foco	Conceptos y reglas del dominio.	Responsabilidades y colaboraciones técnicas.
Resultado	Modelo conceptual o de dominio.	Clases de diseño, interfaces y secuencias.

Vistas del sistema

Estática: conceptos y relaciones. En el enfoque atribuido a Bataglia, un concepto reúne **símbolo** (nombre: Alumno), **intención** (significado) y **extensión** (conjunto de instancias).

Dinámica: comportamiento. Un **DSS** representa al sistema como caja negra y muestra eventos del actor y respuestas. Una secuencia de diseño detalla mensajes entre objetos internos.

Método de resolución de enunciados

1. Identificar objetivos, restricciones y reglas de negocio.
2. Subrayar sustantivos: candidatos a clases, atributos o valores.
3. Subrayar verbos: candidatos a operaciones o relaciones.
4. Depurar: no todo sustantivo merece una clase.
5. Definir responsabilidades, asociaciones y multiplicidades.
6. Elegir patrones por el problema que resuelven.
7. Dibujar UML y después implementar en C#.
8. Comprobar coherencia entre enunciado, modelo, código y pruebas.

Ejemplo de refinamiento

Enunciado	Decisión de análisis / diseño
Formas simples y grupos que se mueven juntos.	Conceptos: Forma y Grupo. Operación: Mover. Estructura recursiva: Composite.
Una estación avisa a distintas pantallas.	Cambio de estado + interesados independientes: Observer.

CONCEPTOS INDISPENSABLES

Análisis = **qué**; diseño = **cómo**. El modelo de dominio no es automáticamente un diagrama de tablas. El patrón se justifica por una necesidad, no por una palabra aislada.

Requisitos y casos de uso

Objetivos · Especificación · Trazabilidad

Requisitos funcionales: servicios y comportamientos. **No funcionales:** cualidades o restricciones, como rendimiento, seguridad y disponibilidad.

Según el enfoque de **Jesús García Molina y colaboradores**, las actividades del negocio ayudan a identificar casos de uso candidatos y la información que circula ayuda a identificar conceptos del dominio. **Cockburn y Korson** orientan los casos de uso hacia objetivos con valor para el actor.

Elemento de especificación	Qué documentar
Identificación	Nombre, objetivo, carátula e historial de revisiones.
Actores y disparador	Roles que interactúan y evento que inicia el caso.
Precondiciones	Condiciones requeridas antes de comenzar.
Flujo principal	Secuencia normal de interacción actor-sistema.
Alternativas y excepciones	Variantes válidas y respuestas ante problemas.
Poscondiciones	Estado o garantías al terminar, según el resultado.
Puntos de extensión	Lugares definidos para incorporar comportamiento adicional.
Reglas y restricciones	Reglas del negocio y requisitos no funcionales asociados.
Artefactos vinculados	Clases, secuencias, DER y prototipos; flujo algorítmico si es necesario.

Ejemplo breve: registrar pedido

Actor: vendedor. **Objetivo:** registrar la compra. **Precondición:** vendedor autorizado. **Disparador:** solicita un nuevo pedido. **Flujo:** identifica cliente, agrega productos, revisa importes y confirma. **Alternativa:** falta stock y se informa el impedimento. **Poscondición exitosa:** pedido registrado con sus ítems.

Plantilla y restricciones

La **plantilla de Coleman** organiza objetivos, actores, condiciones y flujos. La **trazabilidad** conecta necesidades → casos de uso → modelos → implementación → validación.

OCL (Object Constraint Language) expresa restricciones con precisión. Ejemplo ilustrativo:
context Pedido inv FechasValidas: self.fechaMaxEntrega > self.fechaCreacion

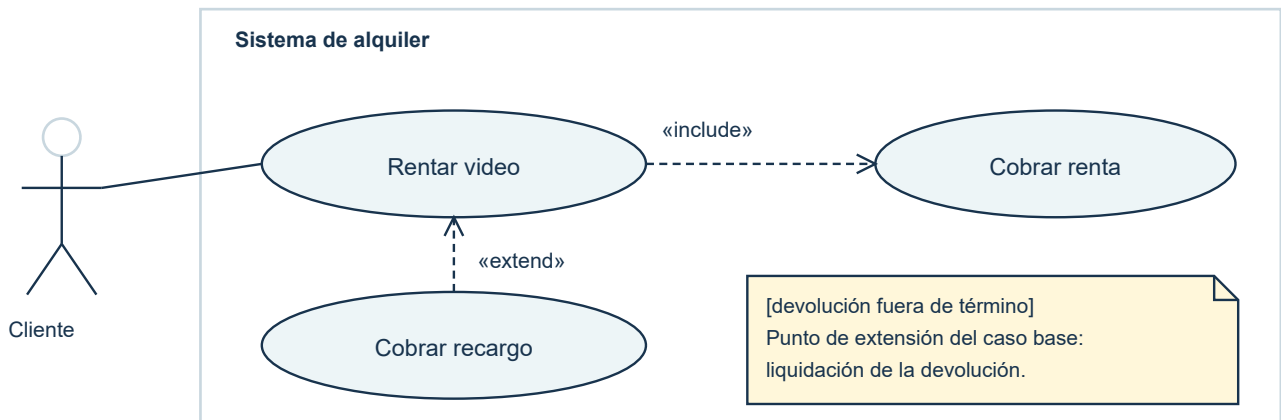
El modelo debe definir los tipos y las operaciones usadas en la restricción. La invariante expresa una condición válida del dominio; no describe por sí sola el algoritmo para cumplirla.

CONCEPTOS INDISPENSABLES

Precondición = situación previa; disparador = inicio; poscondición = garantía final. Un flujo alternativo no es necesariamente un error. No convertir cada paso mínimo en un caso de uso.

Casos de uso y actividades

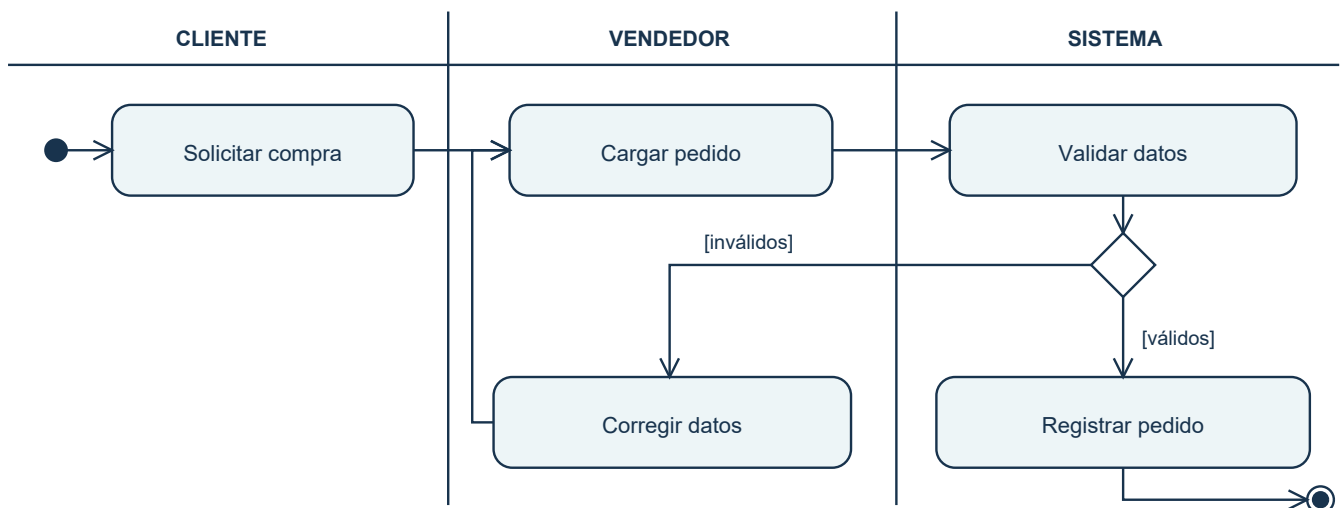
Diseños UML anotados · Ejemplos didácticos



Relación	Dirección y significado
«include»	Base → incluido. Incorpora comportamiento requerido en el punto de inclusión.
«extend»	Extensión → base. Agrega comportamiento bajo una condición; el caso base conserva sentido por sí mismo.

Un caso incluido no necesita ser un objetivo autónomo. El ejemplo de recargo supone que el caso base contempla la liquidación de la devolución. **No usar include/extend para representar simplemente orden temporal.**

Actividad con calles: registro de un pedido

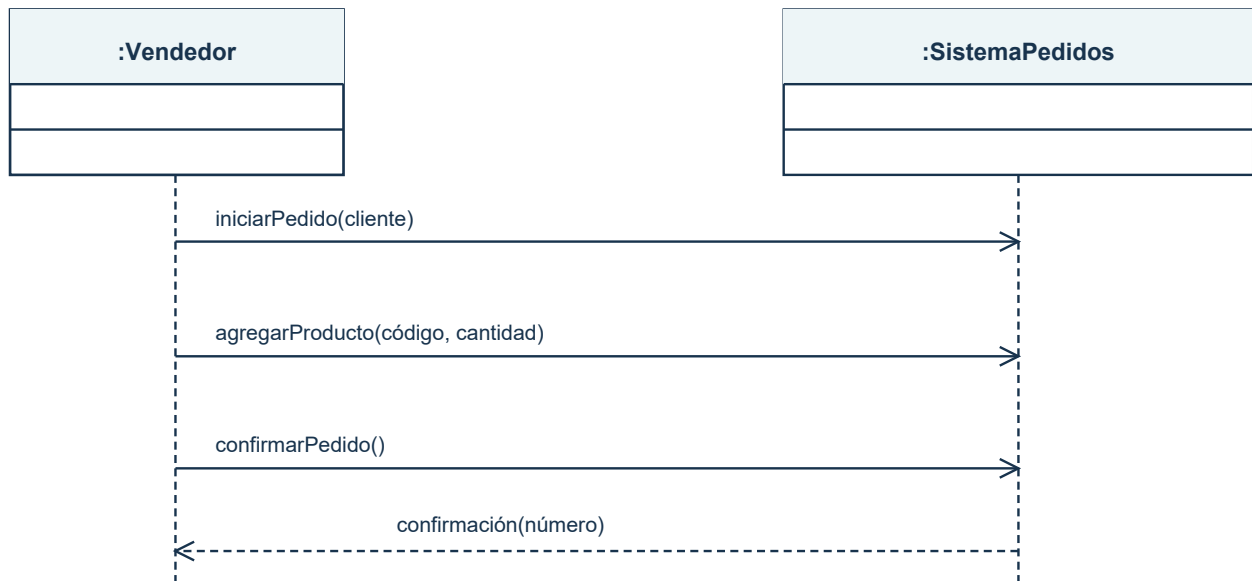


CONCEPTOS INDISPENSABLES

Las calles asignan responsabilidades. El rombo representa una decisión y las condiciones van entre corchetes. El nodo inicial es un círculo lleno; el final, un círculo concéntrico.

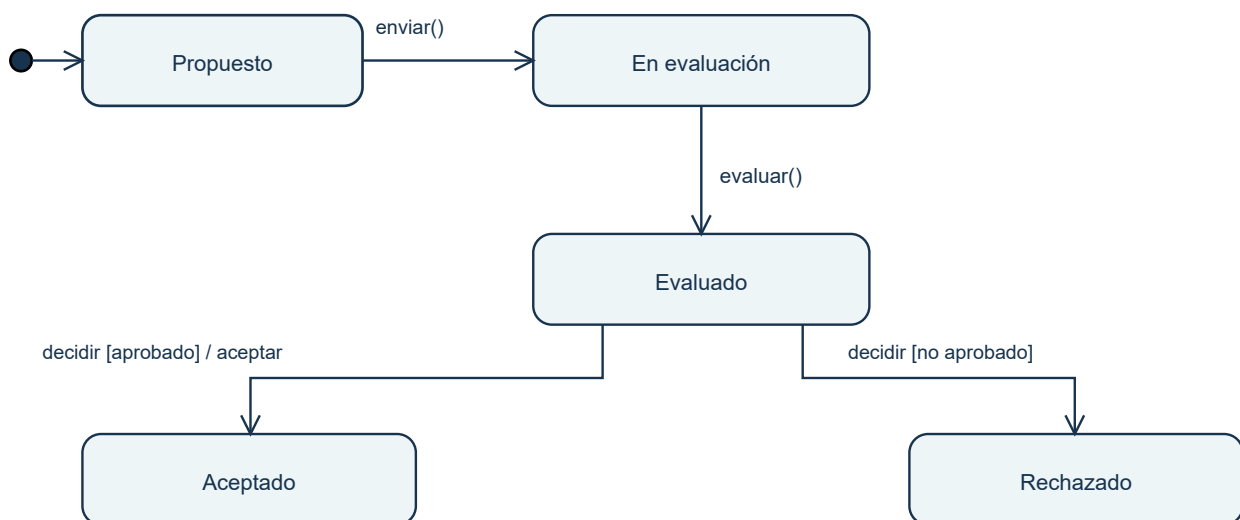
Secuencias y estados

DSS como caja negra · Ciclo de vida de un pedido



Lectura: el tiempo avanza hacia abajo. Cada línea vertical discontinua es una línea de vida. Los mensajes representan operaciones del sistema; la respuesta se muestra con línea discontinua. En este DSS no se exponen repositorios, entidades ni ventanas internas.

Máquina de estados de Pedido



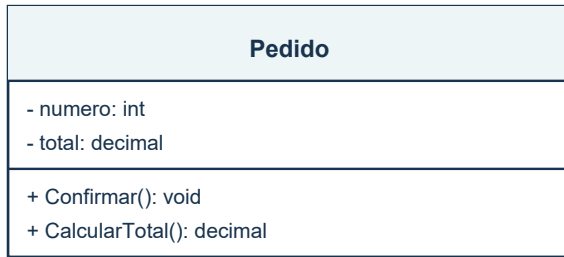
Notación de transición: evento [guarda] / efecto. El evento dispara la evaluación, la guarda habilita la transición y el efecto ejecuta una acción. Los estados y transiciones concretos deben responder a las reglas del negocio.

CONCEPTOS INDISPENSABLES

Secuencia = quién envía mensajes y en qué orden. Estados = cómo cambia la situación de un objeto. Un DSS muestra al sistema como una sola caja negra; una secuencia de diseño abre esa caja.

Diccionario visual UML

Clases · Interfaces · Conectores y notas



Tres compartimentos: nombre, atributos, operaciones. + público; - privado.
 Subrayado: miembro estático.
 Cursiva o {abstract}: elemento abstracto.

Asociación navegable



Dependencia



Generalización



Realización de interfaz



Agregación compartida



Composición



En las dos relaciones todo-parte, A es el todo.
 En generalización, B es la clase base.
 En realización, B es la interfaz.

Conector	Interpretación
Asociación	Vínculo estructural. La navegabilidad indica acceso desde un extremo al otro.
Dependencia	Uso o dependencia; un parámetro o variable local es un ejemplo frecuente.
Generalización / realización	Especialización de una base / implementación de un contrato.
Agregación / composición	Todo-parte débil / propiedad fuerte y exclusiva de la parte a la vez.

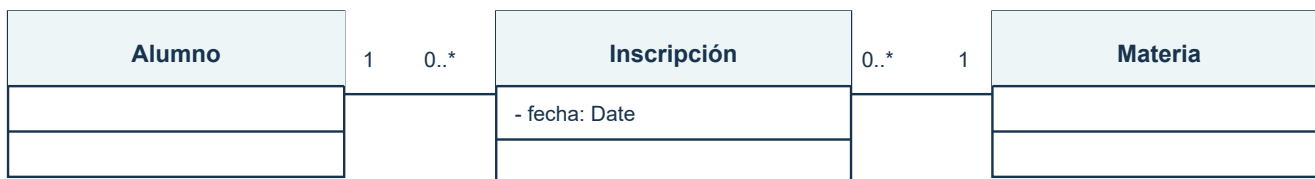
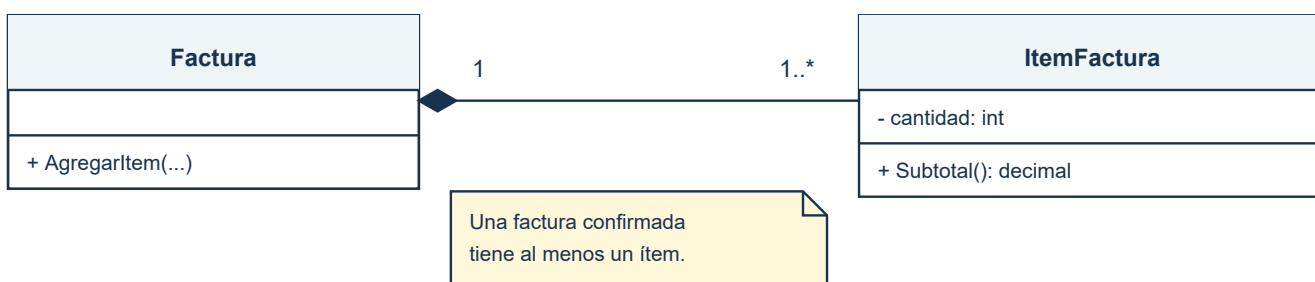
CONCEPTOS INDISPENSABLES

El rombo siempre se coloca en el **todo**. El triángulo apunta a la **base o interfaz**. Una nota UML tiene esquina doblada y puede unirse al elemento con una línea discontinua.

Multiplicidades y persistencia

Cómo leer ambos extremos y trasladar el modelo a datos

UML	Significado	C# habitual
1 o 1..1	Exactamente uno.	Referencia obligatoria validada.
0..1	Ninguno o uno.	Referencia opcional.
* o 0..*	Cero o muchos.	Colección que puede estar vacía.
1..*	Uno o muchos.	Colección con mínimo de uno validado.



Lectura: cada ítem pertenece a una factura; una factura confirmada contiene uno o más ítems. Cada inscripción vincula un alumno con una materia; cada alumno o materia puede tener cero o muchas inscripciones.

Muchos a muchos: se escribe una multiplicidad de muchos en cada extremo, no el rango `*..*`. La entidad asociativa permite guardar datos de la relación. Sus claves foráneas pueden ser simples; su clave primaria puede ser compuesta o propia.

DER en los apuntes: modalidad = mínimo; cardinalidad = máximo. En pata de gallo: círculo = cero, barra = uno, pata de gallo = muchos. Clase persistente → tabla; atributo → columna; identidad → clave primaria; asociación → clave foránea o tabla intermedia.

Composición no equivale a recolección de memoria: crear una lista en el constructor no impone exclusividad. En .NET, el recolector libera objetos inalcanzables; borrar conceptualmente el todo no implica destrucción inmediata de sus partes.

CONCEPTOS INDISPENSABLES

Fijar un objeto y leer cuántos del extremo opuesto puede tener; después invertir la pregunta. Navegabilidad no es multiplicidad. `List<T>` por sí sola no garantiza `1..*`.

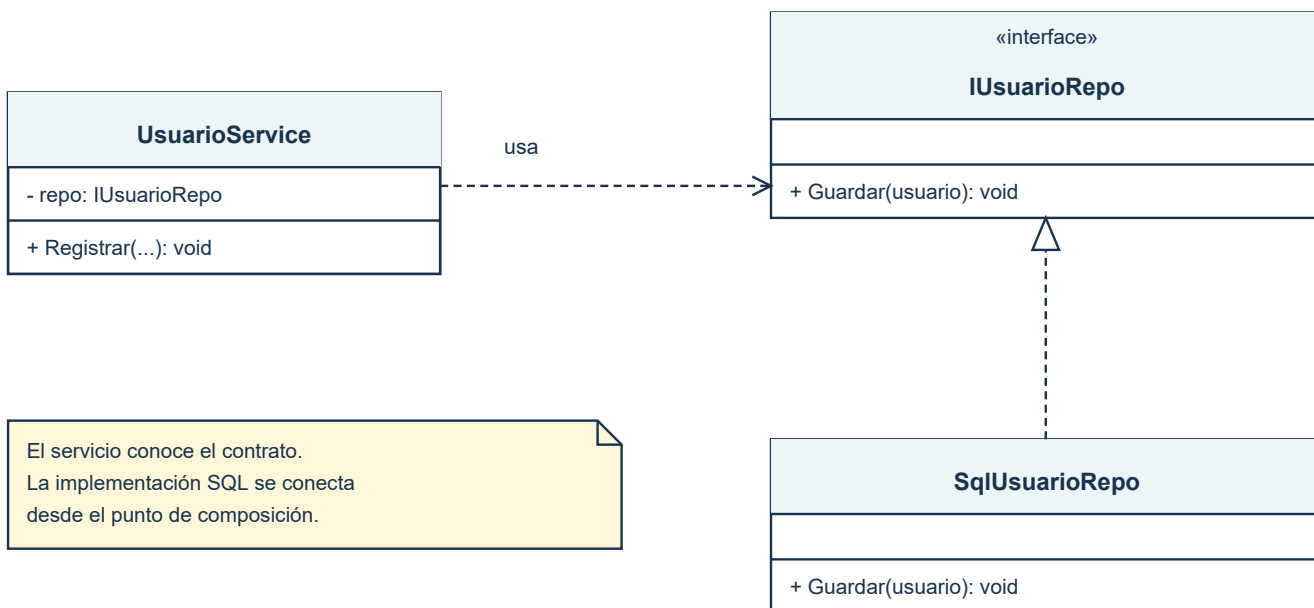
Principios SOLID

Responsabilidades coherentes y dependencias controladas

Principios difundidos por **Robert C. Martin**. Favorecen mantenibilidad y flexibilidad. **Cohesión** es la relación entre responsabilidades internas; **acoplamiento**, la dependencia entre componentes.

Principio	Regla	Ejemplo
S · SRP	Una razón coherente para cambiar.	Separar Pedido de PedidoRepository.
O · OCP	Extender variantes previstas sin reescribir el núcleo estable.	Agregar una implementación de IDescuento.
L · LSP	Un subtipo respeta el contrato de su base.	No exigir Volar() a un Pingüino.
I · ISP	No depender de métodos que no se utilizan.	Separar IImpresora de IFax.
D · DIP	Alto y bajo nivel dependen de abstracciones.	Servicio recibe IRepositoryo.

DIP en UML



Inyección de dependencias: suministrar la dependencia desde fuera, por ejemplo mediante el constructor. Es una técnica; DIP es el principio. Crear objetos con `new` no viola automáticamente DIP: importa quién conoce la implementación concreta.

LSP: no reforzar indebidamente requisitos ni debilitar garantías del contrato. Una subclase que lanza “no implementado” para una operación prometida puede invalidar la sustitución.

CONCEPTOS INDISPENSABLES

SRP no significa un solo método. OCP no prohíbe corregir errores. Usar interfaces no garantiza por sí solo cumplir SOLID. Las variantes deben preservar el contrato.

Patrones de diseño

Concepto · Autores · Clasificación y reconocimiento

Un patrón describe una **solución reutilizable a un problema recurrente en un contexto**. Debe explicarse por nombre, problema y contexto, solución (participantes y relaciones) y consecuencias (beneficios y costos).

Origen: Christopher Alexander, desde la arquitectura edilicia. El **GoF** está formado por Erich Gamma, Richard Helm, Ralph Johnson y John Vlissides; su catálogo reúne **23 patrones**. Los apuntes citan 1995; la ficha editorial del libro registra publicación el 31/10/1994.

Categoría	Problema	Patrones del material
Creacionales	Cómo crear objetos.	Singleton, Abstract Factory, Factory Method.
Estructurales	Cómo combinar clases y objetos.	Composite, Adapter, Decorator.
Comportamiento	Cómo repartir tareas y coordinar.	Observer, Iterator, Mediator, Memento, Interpreter, Template Method.

Los patrones basados en **clases** usan principalmente herencia; los basados en **objetos**, composición y delegación.

Necesidad	Patrón / idea decisiva
Configuración con instancia controlada	Singleton: una instancia y punto de acceso.
Productos compatibles por marca, región o estilo	Abstract Factory: familia o kit.
Subclases deciden qué producto instanciar	Factory Method: creación delegada.
Grupos anidados y tratamiento uniforme	Composite: árbol.
Múltiples interesados reaccionan a cambios	Observer: suscripciones.
Interfaz existente incompatible	Adapter: convertir la interfaz.
Agregar capacidades mediante envolturas	Decorator: mismo contrato y nuevas responsabilidades.

Otros patrones que conviene reconocer

Iterator: recorrer una colección sin exponer su estructura. **Mediator:** centralizar coordinación y reducir referencias cruzadas. **Memento:** capturar y restaurar estado preservando encapsulamiento. **Interpreter:** representar una gramática e interpretar expresiones. **Template Method:** fijar el esqueleto de un algoritmo y especializar algunos pasos.

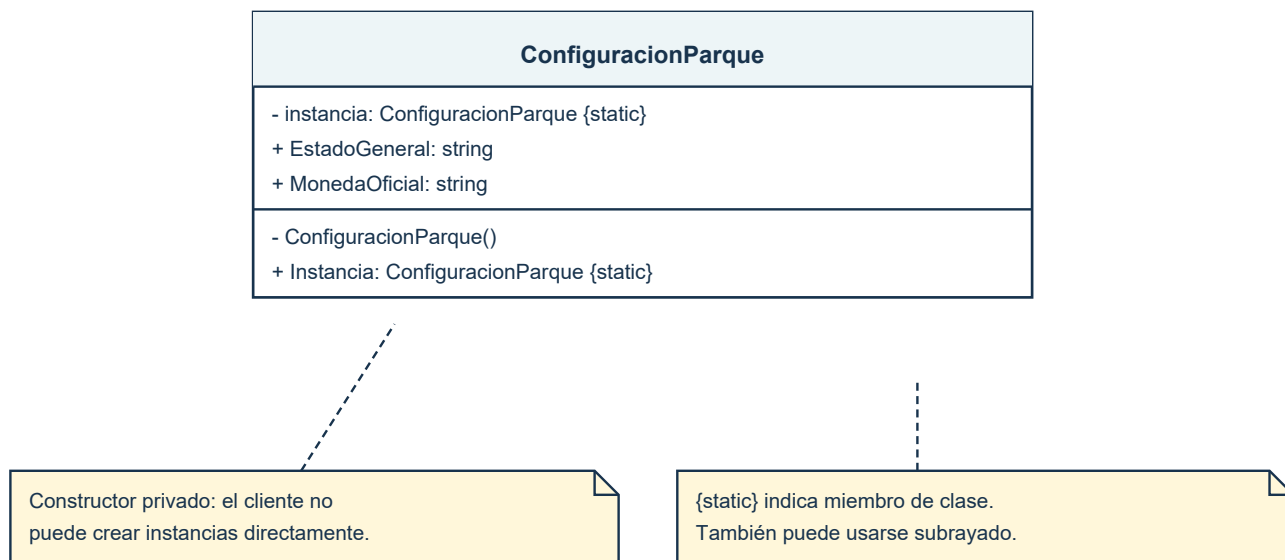
CONCEPTOS INDISPENSABLES

Un patrón no es código para copiar mecánicamente. En examen: explicar la necesidad, identificar participantes, dibujar relaciones y justificar beneficios y costos.

Singleton

Patrón creacional · Configuración compartida

Propósito: controlar una única instancia dentro de su ámbito y proporcionar acceso a ella. Evita configuraciones duplicadas; el acceso global puede dificultar aislamiento y pruebas.



Implementación breve en C#

```

public sealed class ConfiguracionParque
{
    private static readonly Lazy<ConfiguracionParque> instancia =
        new Lazy<ConfiguracionParque>(() => new ConfiguracionParque());
    private ConfiguracionParque() {}
    public static ConfiguracionParque Instancia => instancia.Value;
    public string EstadoGeneral { get; set; } = "Abierto";
}
  
```

El ejemplo requiere **System**. `Lazy<T>` con su configuración predeterminada protege la inicialización. **No protege las escrituras concurrentes de EstadoGeneral**. La sincronización de los datos mutables se diseña por separado.

Anotaciones para justificar

- Una instancia compartida centraliza la configuración dentro de su ámbito; no garantiza una única instancia entre procesos o servidores distintos.
- Puede implementar interfaces. La variante sealed con constructor privado no permite herencia ordinaria.
- Un pool de varias instancias deja de ser un Singleton estricto.

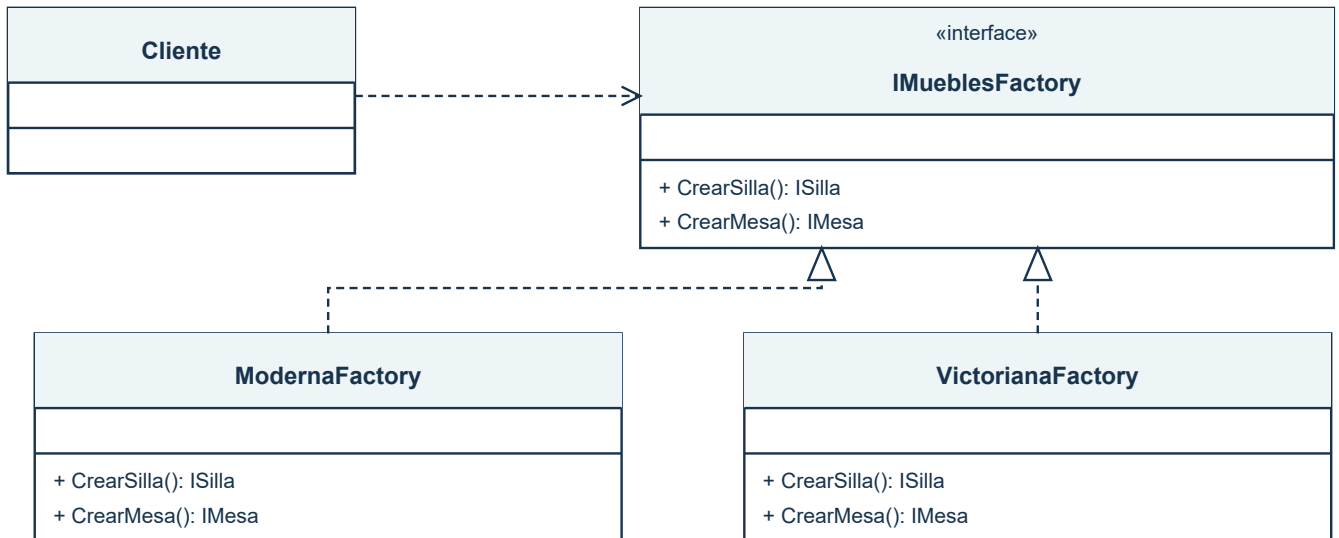
CONCEPTOS INDISPENSABLES

Constructor privado + acceso estático + instancia controlada. **Inicialización segura no equivale a estado compartido seguro.**

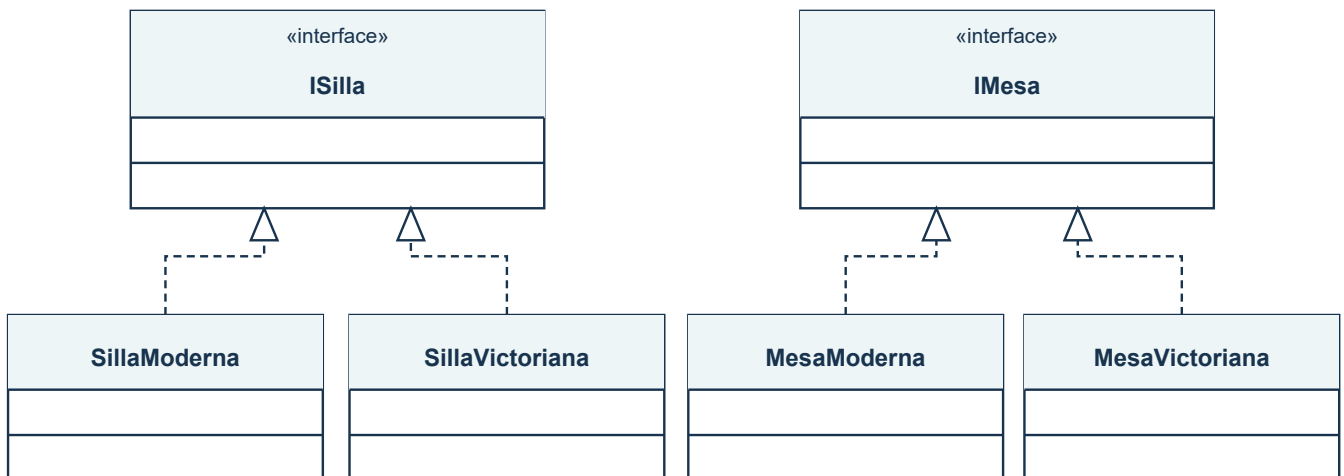
Abstract Factory

Patrón creacional · Familias de productos compatibles

Problema: crear familias relacionadas sin acoplar al cliente a productos concretos. Ejemplo: muebles modernos o victorianos; cada familia crea una silla y una mesa compatibles.



ModernaFactory crea las dos variantes modernas; VictorianaFactory, las victorianas.



Participantes: Cliente; AbstractFactory (IMueblesFactory); ConcreteFactory (cada fábrica); AbstractProduct (ISilla e IMesa); ConcreteProduct (las cuatro variantes).

Ventaja: cambiar la familia completa y mantener consistencia. **Costo:** añadir un nuevo tipo de producto (por ejemplo, sofá) obliga a ampliar el contrato y todas las fábricas.

Otros ejemplos: pizzería argentina/italiana con pizza y empanada; autos lujo/estándar con sedán y SUV. Cada producto concreto realiza su interfaz; su fábrica conoce la clase concreta que instancia.

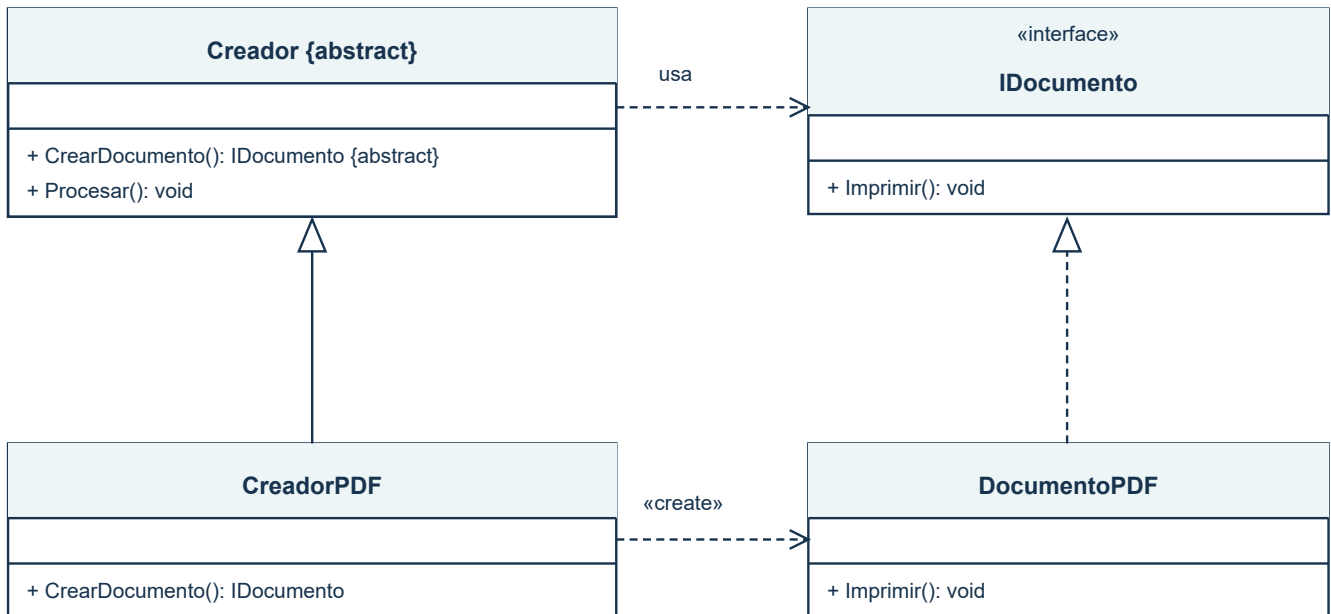
CONCEPTOS INDISPENSABLES

Reconocer “familia”, “kit” y “compatibilidad”. Agregar una **familia** suele ser más sencillo que agregar un **tipo de producto**.

Factory Method

Patrón creacional · Instanciación delegada a subclases

Propósito: definir un punto de creación que las subclases implementan. La lógica del creador utiliza el producto mediante su contrato.



Procesar() llama a CrearDocumento() y luego a Imprimir().
El creador concreto decide qué implementación devolver.

Factory Method	Abstract Factory
Especializa un punto de creación mediante subclases.	Agrupar operaciones para crear una familia.
Se reconoce por delegación de instanciación.	Se reconoce por compatibilidad entre productos.
Ejemplo: CreadorPDF devuelve DocumentoPDF.	Ejemplo: ModernaFactory crea silla y mesa modernas.

Ventaja: agregar un nuevo producto mediante un creador nuevo sin cambiar la lógica estable de procesamiento. **Costo:** proliferación de subclases.

Ejemplos alternativos: Logistica.CrearTransporte() y LogisticaCamion; Notificador.CrearMensaje() con implementaciones Email y SMS.

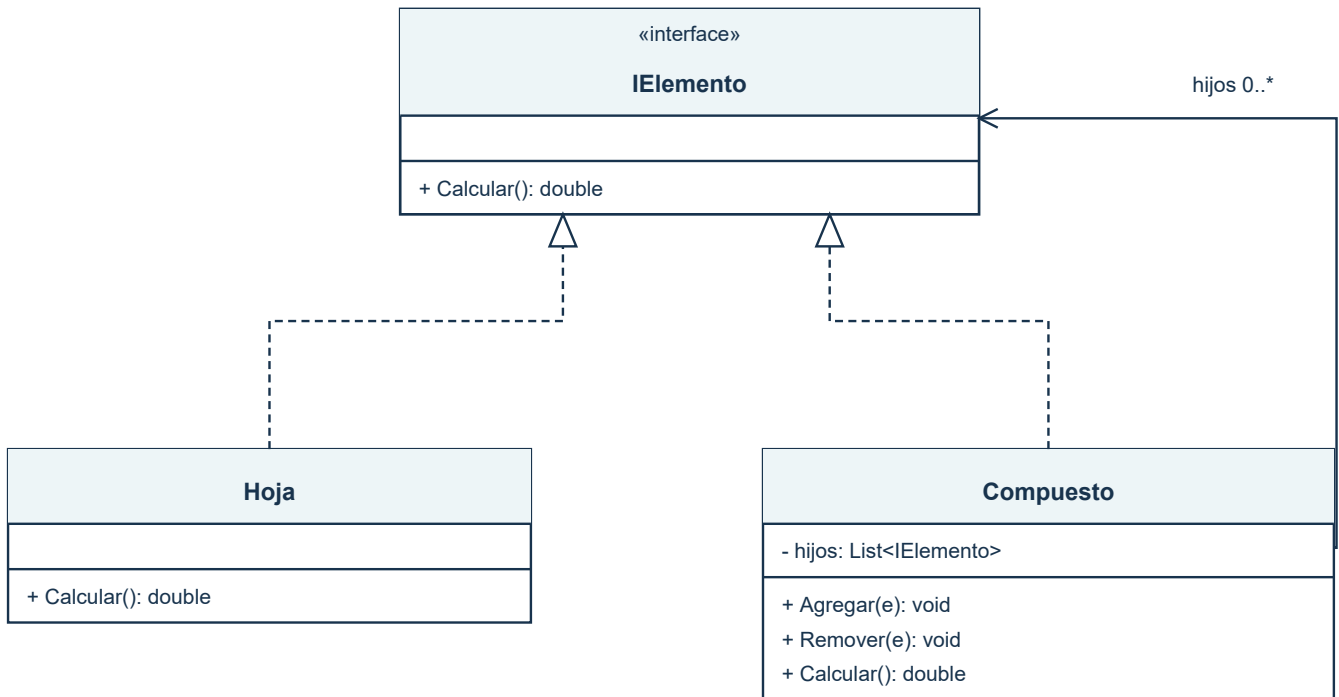
CONCEPTOS INDISPENSABLES

Una fábrica simple con un switch no es automáticamente Factory Method. La clave es que **las subclases deciden la creación**.

Composite

Patrón estructural · Jerarquía recursiva parte-todo

Propósito: tratar hojas y grupos mediante una interfaz común. El compuesto contiene componentes, que pueden ser simples o a su vez compuestos.



La asociación recursiva termina en **IElemento**, no solamente en **Hoja**.
`Calcular()` en el compuesto suma `Calcular()` de cada hijo.

Rol	Ejemplo en archivos	Ejemplo en parque
Componente	IElemento	IElementoParque
Hoja	Archivo	Atraccion
Compuesto	Carpeta	ZonaCompuesta

Fórmula: $\text{Métrica}(\text{compuesto}) = \sum \text{Métrica}(\text{hijo})$. La hoja devuelve su propio valor; el compuesto acumula recursivamente. El cliente invoca una operación uniforme sin preguntar por el tipo concreto.

Ventaja: simplificación del cliente y extensión de jerarquías. **Costos:** impedir ciclos, controlar hijos permitidos y establecer reglas de pertenencia.

Anotación: `Agregar` y `Remover` pueden estar solo en el compuesto. Composite no exige que las hojas implementen operaciones inaplicables. El patrón tampoco obliga siempre a usar composición UML: el rombo depende de la propiedad y del ciclo de vida.

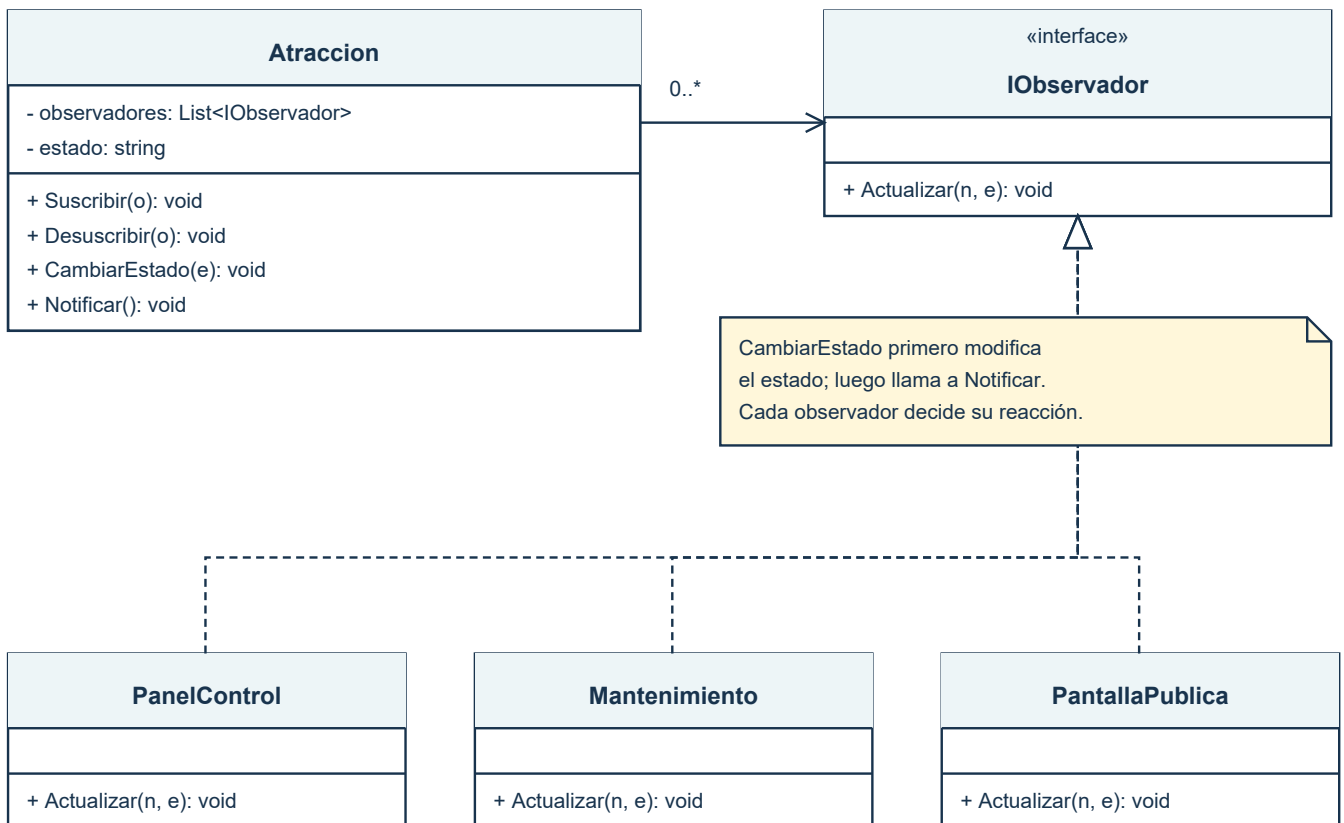
CONCEPTOS INDISPENSABLES

“Árbol + mismo contrato” identifica Composite. **Composite** es un patrón; **composición** es una relación UML.

Observer

Patrón de comportamiento · Suscripciones y notificaciones

Propósito: notificar un cambio a múltiples interesados sin conocer sus clases concretas. El sujeto depende del contrato de observador.



Secuencia: suscribir → modificar estado → notificar → ejecutar Actualizar en cada receptor. Las operaciones también aparecen como Attach, Detach, Notify y Update.

Ventaja: una app móvil nueva implementa IObservador y se suscribe sin cambiar la lógica del sujeto. **Costos:** cascadas de actualizaciones, orden de avisos, manejo de errores y suscripciones que no se eliminan.

Corrección frecuente: hay bajo acoplamiento con clases concretas, no acoplamiento cero. Observer no garantiza por sí mismo asincronía ni incorporación de componentes sin recompilar o desplegar.

Otros ejemplos: precios o stock de un producto; sensor de temperatura y alarmas; estación meteorológica con pantalla de TV y app móvil.

CONCEPTOS INDISPENSABLES

La colección de observadores pertenece al **sujeto**. Notificar recorre esa colección e invoca el contrato de actualización. El dominio no necesita crear ventanas concretas.

Arquitectura en siete capas

Modelo de los apuntes · Responsabilidades y contratos

Capa	Responsabilidad
UI	Capturar datos y presentar resultados: consola, WinForms, MDI, login, botones y grillas.
Aplicación	Orquestrar casos de uso mediante servicios de aplicación.
Servicios	Capacidades transversales: seguridad, sesión y registro.
Dominio	Entidades y reglas de negocio.
Repositorio	Encapsular y coordinar persistencia.
Contextos	Acceso específico a SQL, NoSQL u otros mecanismos.
Abstracciones	Interfaces compartidas para invertir dependencias.

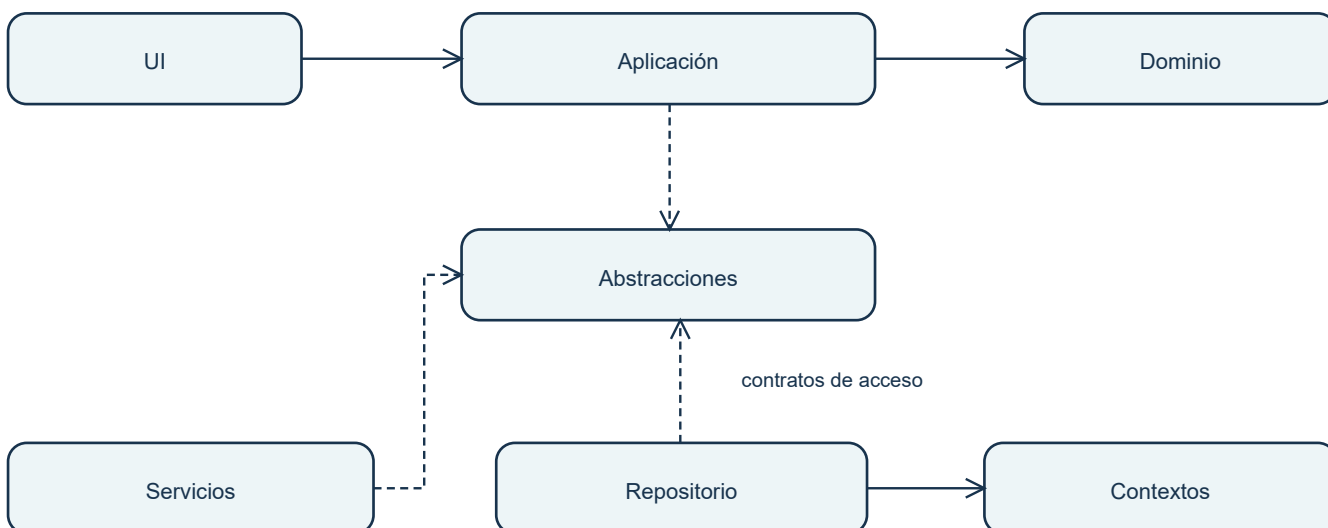


Diagrama de responsabilidades orientativo: las flechas muestran usos principales, no una única cadena obligatoria de todas las dependencias. Las implementaciones concretas se conectan mediante contratos.

Sesión: ManejadorSesion reside en Servicios y trabaja con IUserario de Abstracciones; la entidad concreta implementa ese contrato. Así no queda ligado a un dominio particular.

UI: el botón llama a loginService.Login(...); no realiza SQL ni concentra reglas del negocio. La consola permite verificar AppService antes de invertir en la GUI.

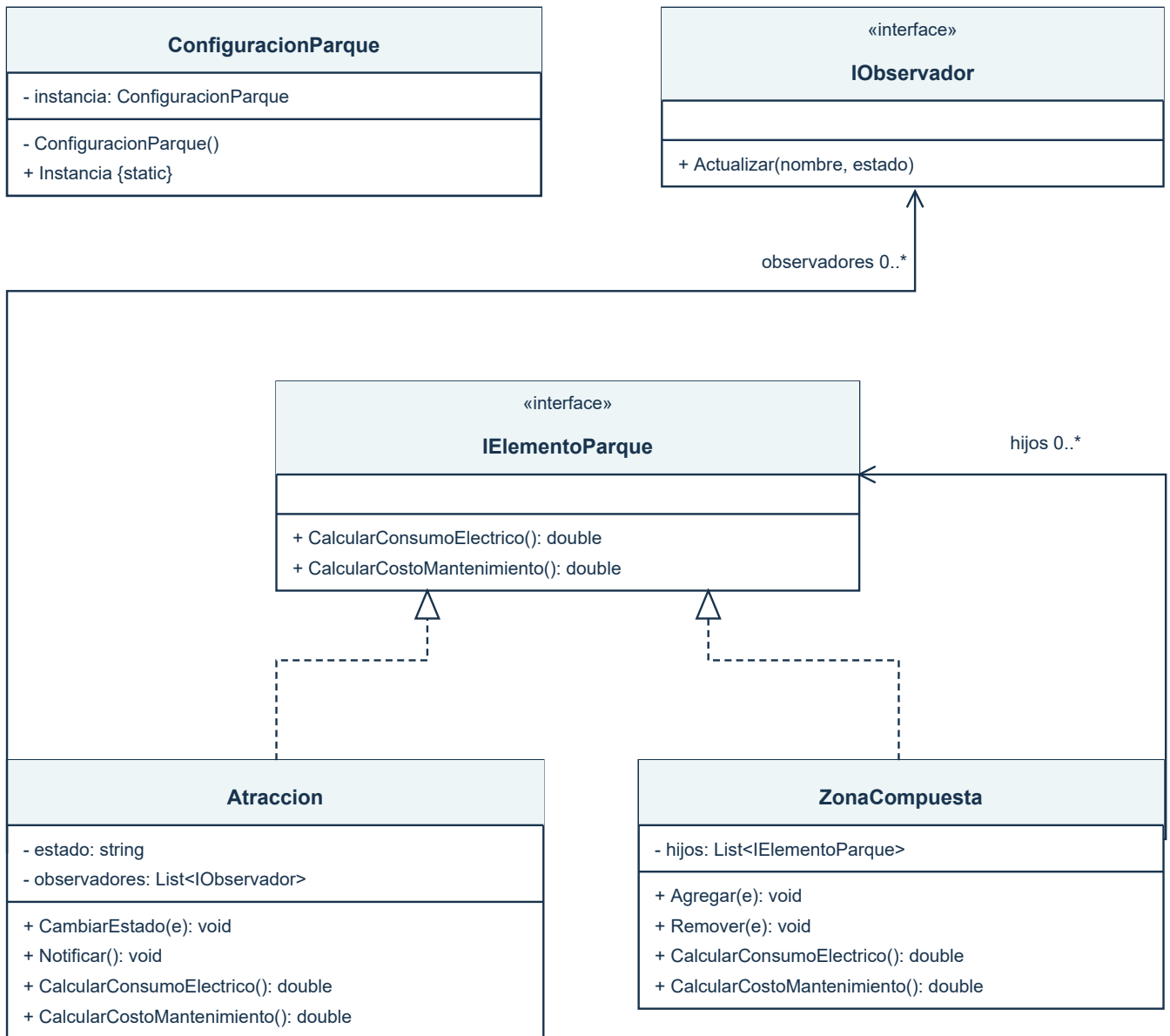
Mapeo manual: un SqlDataReader se transforma en Usuario asignando Id y Username. Tipos, nulos y validaciones se resuelven en persistencia; no en la pantalla.

CONCEPTOS INDISPENSABLES

UI presenta; aplicación coordina; dominio define reglas; persistencia encapsula almacenamiento. Las siete capas son el modelo de este material, no una obligación universal.

Wonderland Tech

Diseño integrado UML · Un objeto puede participar en varios patrones



Atraccion es Leaf de Composite y Subject de Observer.
 ZonaCompuesta contiene elementos; Atraccion contiene observadores.
 PanelControl, Mantenimiento y PantallaPublica realizan IObservador.

CONCEPTOS INDISPENSABLES

Configuración → Singleton. Jerarquía de zonas → Composite. Avisos de estado → Observer. El diseño conserva separados los tres problemas y combina sus contratos.

Wonderland en funcionamiento

Cálculos · Flujo de avisos · Correspondencia con C#

Necesidad	Implementación esencial
Singleton	Constructor privado, almacenamiento estático y punto de acceso.
Composite	List<IElementoParque>; hoja devuelve valor; compuesto recorre hijos y acumula.
Observer	List<IObservador>; alta, baja y notificación por contrato.

Árbol de ejemplo

Parque contiene Zona Aventura y Carrusel. **Zona Aventura** contiene Montaña Rusa y Barco Pirata. Llamar a la raíz propaga el cálculo por ramas y hojas.

Potencia(zona) = Σ Potencia(hijo)

Costo(zona) = Σ Costo(hijo)

Datos de cada versión	Zona Aventura	Parque	Costo total
120 / 30 / 65 kW Costos: 5.000 / 1.200 / 2.300	185 kW	215 kW	8.500
150,5 / 45 / 80 kW Costos: 3.000 / 800 / 1.200	230,5 kW	275,5 kW	5.000

Los valores siguen el orden montaña rusa / carrusel / barco. Son **dos ejemplos distintos** de los PDF; no deben mezclarse. kW es potencia; para energía con potencia constante, $E = P \times t$ (kWh si t está en horas).

Notificación de fuera de servicio

1. CambiarEstado asigna el nuevo estado de Atraccion.
2. Notificar recorre los observadores registrados.
3. PanelControl actualiza la información operativa.
4. Mantenimiento genera la alerta si corresponde.
5. PantallaPublica informa al visitante.

Justificación esperada

Singleton: centraliza configuración; la sincronización del estado es un problema adicional. **Composite:** evita condicionales de tipo en el cliente. **Observer:** permite sumar receptores sin introducir conocimiento de sus clases en la atracción.

Errores que invalidan el diseño

Guardar observadores en la zona sin una necesidad explícita; relacionar el compuesto únicamente con hojas; omitir bajas de suscripción; crear pantallas concretas desde el dominio; suponer que List<T> valida las reglas de multiplicidad.

CONCEPTOS INDISPENSABLES

Las interfaces conectan responsabilidades. La integración correcta conserva el significado de cada relación: **hijos del compuesto** y **suscriptores del sujeto** son colecciones diferentes.

Glosario de repaso

Definiciones breves para recuperación activa

Término	Definición
Abstracción	Representación que conserva lo relevante y omite detalles innecesarios.
Acoplamiento	Grado de dependencia entre componentes.
Cohesión	Relación entre responsabilidades internas de un componente.
Bache semántico	Distancia entre el problema y su representación en software.
Modelo de dominio	Conceptos, relaciones y reglas del negocio.
Caso de uso	Interacciones para alcanzar un objetivo de un actor.
DSS	Secuencia entre actores y sistema tratado como caja negra.
Precondición / poscondición	Condición previa / garantía al finalizar.
Invariante	Restricción que se mantiene en los estados válidos del modelo.
Trazabilidad	Vínculos entre requisitos, diseño, implementación y validación.
Multiplicidad	Número permitido de instancias relacionadas en un extremo.
Navegabilidad	Posibilidad de acceder desde un objeto a otro.
Polimorfismo	Uso de un contrato común con implementaciones diferentes.
Encapsulamiento	Control del acceso al estado y a la implementación interna.
OCL	Lenguaje de restricciones sobre modelos.
DIP	Principio de dependencias hacia abstracciones.
Inyección de dependencias	Suministro externo de las dependencias de un objeto.
Repositorio	Abstracción de operaciones de persistencia.
ORM	Mapeo entre objetos y datos relacionales.
Thread-safe	Seguro para uso concurrente bajo garantías especificadas.
Guarda	Condición que habilita una transición o camino.
Realización	Relación entre una implementación y el contrato que cumple.
Composición	Relación de propiedad fuerte de una parte por un todo.

CONCEPTOS INDISPENSABLES

Estudiar definiendo el término sin mirar; después contrastar con la tabla. Para consolidar cada definición, agregar un ejemplo y un contraejemplo.

Autoevaluación

Diez preguntas con respuestas breves

01 ¿Cuál es la diferencia entre análisis y diseño OO?

Respuesta: El análisis identifica conceptos y necesidades; el diseño define objetos de software, responsabilidades y colaboraciones.

02 ¿Cuáles son las cinco actividades estructurales de Pressman?

Respuesta: Comunicación, planeación, modelado, construcción y despliegue. Su organización no tiene que ser siempre lineal.

03 ¿En qué difieren un modelo iterativo y uno incremental?

Respuesta: El iterativo refina en ciclos; el incremental agrega funcionalidad mediante entregas. Pueden combinarse.

04 ¿Hacia dónde apuntan include y extend?

Respuesta: Include: desde el caso que incluye al incluido. Extend: desde la extensión al caso base.

05 ¿Cómo se representa muchos a muchos y cómo se lleva al DER?

Respuesta: Con multiplicidades de muchos en ambos extremos. Se resuelve con una entidad o tabla asociativa que referencia ambas entidades.

06 ¿Qué distingue Abstract Factory de Factory Method?

Respuesta: Abstract Factory crea familias compatibles; Factory Method delega un punto de creación a subclases.

07 ¿Por qué Composite simplifica el cálculo de una zona?

Respuesta: Hoja y compuesto comparten un contrato. El compuesto acumula recursivamente las métricas de los hijos sin que el cliente distinga tipos.

08 ¿Cómo se agrega una app móvil como observador?

Respuesta: Implementa IObservador y se suscribe al sujeto. La atracción no necesita conocer su clase concreta.

09 ¿Una inicialización segura de Singleton protege todas sus propiedades?

Respuesta: No. La creación de la instancia y el acceso concurrente al estado son problemas diferentes.

10 ¿Cómo se aplican SOLID y las capas al inicio de sesión?

Respuesta: UI captura entradas; aplicación coordina; servicios y dominio aplican reglas; persistencia usa contratos. SRP separa responsabilidades y DIP evita depender de detalles innecesarios.

CONCEPTOS INDISPENSABLES

Una respuesta sólida incluye definición, diferencia con un concepto cercano y aplicación. En patrones, sumar participantes y al menos una consecuencia.

Repaso antes del examen

Lista de control · Técnica de estudio · Referencias

Control de diagramas

- Rombo del lado del todo; triángulo hacia la clase base o interfaz.
- Línea continua para generalización; discontinua para realización.
- Multiplicidades leídas desde ambos extremos; obligatoriedad justificada.
- Colección del Composite tipada con el componente común.
- Lista de Observer en el sujeto; contrato de actualización explícito.
- Mensajes de secuencia ordenados y coherentes con las operaciones.
- Estados, guardas y efectos compatibles con las reglas de negocio.
- Notas diferenciadas de clases; dependencias y asociaciones no intercambiadas.

Control de implementación

Colecciones inicializadas; estado encapsulado; reglas de cardinalidad validadas; persistencia fuera de UI; dependencias suministradas por contratos; en Observer, actualizar estado antes de notificar; en Singleton, no confundir inicialización con sincronización de propiedades.

Método de repaso

1. Reconstruir de memoria las tablas. 2. Comparar conceptos próximos. 3. Resolver un enunciado nuevo con justificación, UML y código. 4. Corregir errores concretos. 5. Repetir con separación temporal, concentrándose en lo que no pudo explicarse.

Documentos base

Guia_Integral_1er_Parcial.docx · Perfecto esto que hiciste pero no te olvides de t.pdf · Necesito que armes un informe resumido pero detal.pdf · hay alguna info que te hayas pasado de alto_.pdf

Referencias técnicas de las precisiones

OMG: UML 2.5.1

Microsoft: Lazy<T> e inicialización

Microsoft: recolección de memoria

GoF: ficha editorial de Design Patterns

Los diagramas se redibujaron como ejemplos de estudio a partir de los conceptos del material. Las notas distinguen convenciones, simplificaciones y condiciones que deben validarse en cada enunciado.

CONCEPTOS INDISPENSABLES

Dominás un tema cuando podés **definirlo, distinguirlo, aplicarlo y justificarlo** sin depender de memorizar una imagen.